

CloudSentinel CSPM - Code, Data, and Outcome Workflow

A code-guided walkthrough of how the application turns cloud API responses into an operator-facing CSPM outcome.

Scope: the current implementation, with AWS as the detailed example. Azure and GCP use the same collector -> normalizer -> persistence spine for the resources currently supported.

What a CISO or security analyst receives

- A tenant-isolated cloud inventory: active assets, raw evidence, normalized posture fields, tags, relationships, and history.
- Prioritized posture findings, correlated attack paths, compliance snapshots, vulnerability context, runtime signals, and DSPM classifications.
- Honest partial-scan visibility: a denied, failed, or truncated cloud API scope is recorded as an error and does not make real assets disappear.

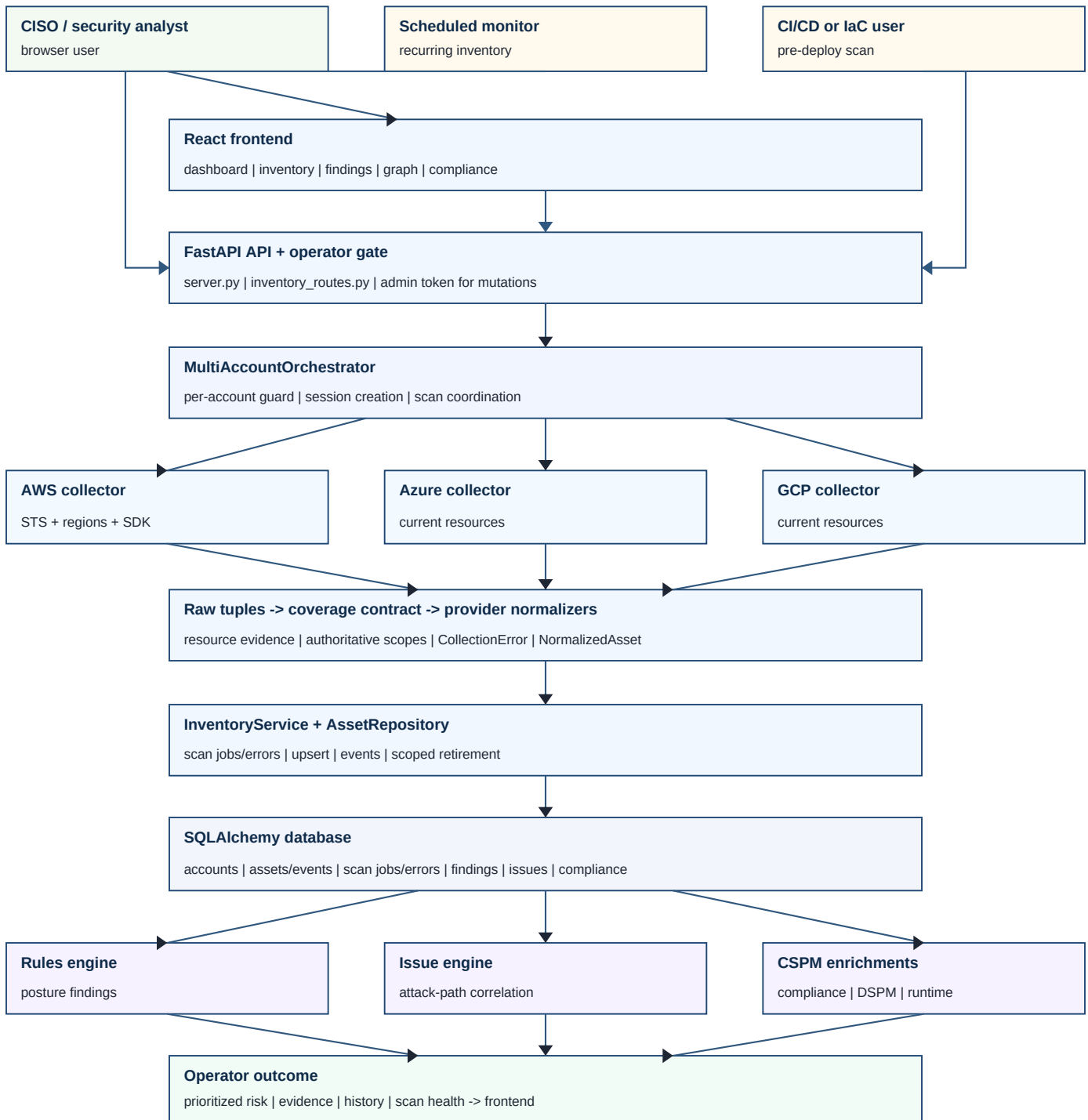
The high-level pipeline

```
Operator / scheduled monitor
-> FastAPI scan endpoint
-> MultiAccountOrchestrator
-> AwsRawCollector (parallel read-only API calls)
-> InventoryService (normalize + tracked scan job)
-> AssetRepository (upsert + asset events + safe retirement)
-> rules -> findings -> attack-path issues -> compliance snapshot
-> frontend dashboard, inventory, graph, reports
```

Project elements and their actual role

- frontend/: CISO/analyst UI. It calls /api endpoints for dashboards, assets, findings, issues, graph, compliance, and onboarding.
- api/inventory_routes.py: HTTP boundary. Validates provider/account/region inputs, starts scans, exposes paginated inventory and scan-job status, and gates privileged mutations.
- inventory/orchestrator.py: Per-account coordinator. Builds an assumed-role session, selects the provider collector, serializes same-account scans, then triggers rules/issues/compliance.
- inventory/aws_raw_collector.py: Read-only AWS discovery. Discovers enabled regions, creates SDK clients, calls selected list/describe APIs in parallel, enriches key resources, and records coverage/errors.
- inventory/normalizers.py: Pure translators. They convert provider-specific dictionaries into one NormalizedAsset contract; they never call AWS.
- inventory/service.py + repository.py: Creates durable scan jobs, isolates bad records, upserts inventory, writes drift events, and safely retires assets only in fully collected scopes.
- inventory/rules.py + issues.py: Evaluates posture checks and correlated attack paths against normalized assets; findings and issues have open/resolved/reopened lifecycles.
- db/models.py + db/base.py: SQLAlchemy schema and engine management. SQLite for local/test, PostgreSQL for production; tenant-scoped asset identity and query indexes.
- inventory/monitor.py: Scheduled end-to-end cycle, optional DSPM scan, and alert-delta processing.

Project-wide architecture: the whole application



The vector diagram above is the rendered project model. The reusable Mermaid source lives in docs/CSPM_PROJECT_ARCHITECTURE.mmd, so it can be embedded in the repository README, GitHub, or a design document.

Read the arrows as data ownership: cloud responses become evidence, evidence becomes account-scoped inventory, inventory produces risk records, and only the query/API layer returns those records to the UI.

1. Scan initiation and account boundary

A user connects a cloud account with a provider, account identifier, and read-only role ARN. The API persists a CloudAccount row; AWS onboarding provides an ExternalId to protect the cross-account AssumeRole trust relationship.

```
POST /api/inventory/accounts/{account_id}/scan
{
  "region": "us-east-1"
}

# endpoint calls
result = _run_account_scan(account_id, body.region)
```

The same account cannot run two scans at the same time in one API process. This prevents one scan from interpreting another scan's still-in-flight resources as missing.

```
with _account_scan_guard(account):
    collector = self._make_collector(account)
    resources = collector.collect()
    scan = self.service.persist_scan(
        account.provider, account.identifier, resources,
        authoritative_scopes=collector.authoritative_scopes,
        collection_errors=collector.collection_errors,
    )
```

Outcome: the scan is attached to exactly one CloudAccount. Data from another AWS account/subscription/project cannot overwrite it because assets are keyed by account_id + provider + resource_id.

2. AWS authentication and regional discovery

For another AWS account, the orchestrator calls sts:AssumeRole using the stored role ARN and ExternalId. For the scanner's own AWS account, it can use the ambient SDK identity only when the caller account matches the target account.

```
creds = sts.assume_role(
    RoleArn=account.role_arn,
    RoleSessionName="odineyes-inventory-scan",
    DurationSeconds=3600,
    ExternalId=account.external_id,
)["Credentials"]

return boto3.Session(
    aws_access_key_id=creds["AccessKeyId"],
    aws_secret_access_key=creds["SecretAccessKey"],
    aws_session_token=creds["SessionToken"],
    region_name=self.region,
)
```

The collector discovers enabled regions with EC2 DescribeRegions. If discovery is unavailable it scans the configured home region instead of silently collecting nothing.

```
resp = ec2.describe_regions(
    Filters=[{"Name": "opt-in-status",
              "Values": ["opt-in-not-required", "opted-in"]}]
)
regions = sorted(r["RegionName"] for r in resp["Regions"])
return regions or [self.region]
```

Outcome: rogue or shadow resources in enabled AWS regions are in scope by default. Global services such as S3 and IAM are collected once; regional services are collected per enabled region.

3. Raw AWS data fetching

The normal scan uses an explicit, posture-relevant registry. The collector creates service clients with adaptive retries, then uses a bounded thread pool for network I/O.

```
OPERATIONS = [
    ("ec2", "describe_instances", {}, "aws.ec2.instance"),
    ("ec2", "describe_security_groups", {}, "aws.ec2.security_group"),
    ("s3", "list_buckets", {}, "aws.s3.bucket"),
    ("iam", "list_roles", {}, "aws.iam.role"),
    ("rds", "describe_db_instances", {}, "aws.rds.db_instance"),
    ("lambda", "list_functions", {}, "aws.lambda.function"),
    ...
]
```

For each operation, the native SDK response is kept as a Python dict. The collector adds Region and performs best-effort enrichment for high-value types such as S3, IAM, Lambda, Secrets Manager, CloudTrail, Config, and KMS.

```
if client.can_paginate(snake_op):
    pages = client.get_paginator(snake_op).paginate(
        PaginationConfig={"MaxItems": 5000}, **kwargs
    )
else:
    pages = [getattr(client, snake_op)(**kwargs)]

for item in extracted_items:
    item["Region"] = region
    out.append((source_type, item))
```

Example: raw S3 resource after collection and enrichment

```
("aws.s3.bucket", {
    "Name": "customer-exports",
    "CreationDate": "2026-07-19T09:12:00Z",
    "Region": "eu-west-1",
    "PolicyStatus": {"IsPublic": true},
    "PublicAccessBlock": {},
    "Encryption": {"enabled": true, "algorithm": "AES256"},
    "Versioning": "Enabled",
    "Tags": {"env": "prod", "owner": "data"}
})
```

Example: raw EC2 instance

```
("aws.ec2.instance", {
    "InstanceId": "i-0ab123",
    "Region": "us-east-1",
    "PublicIpAddress": "198.51.100.20",
    "VpcId": "vpc-01", "SubnetId": "subnet-01",
    "SecurityGroups": [{"GroupId": "sg-01"}],
    "State": {"Name": "running"}
})
```

Outcome: raw evidence is preserved. Later normalizers and rules work from a stable collected record, while API-specific details remain available for investigation.

4. Collection completeness and error handling

A CSPM must not treat an AccessDenied, timeout, client construction failure, or 5,000-item safety cap as an empty cloud response. The collector records a CollectionError and only marks successfully completed type/region scopes as authoritative.

```
resources, complete = future.result()
out.extend(resources)
if complete:
    authoritative_scopes.add((source_type, scope_region))
else:
    collection_errors.append(CollectionError(
        source_type=source_type,
        operation=f"{service}.{operation}",
        region=scope_region,
        message="result exceeded the 5000-item safety limit",
    ))
```

For S3, a global successful list is authoritative for all bucket regions. For EC2, a successful DescribeInstances in eu-west-1 is authoritative only for EC2 instance assets in eu-west-1.

Outcome: if an account lacks permission to list RDS in one region, existing RDS inventory there stays active and existing findings remain honest. The scan is marked partial with a stored scan error rather than silently resolving risk.

5. Normalization: raw provider data -> common asset contract

Normalizers are pure functions selected by source type. They extract the common posture fields that rules can query across clouds: identity, region, tags, public exposure, encryption, properties, and relationships.

```
def normalize_s3_bucket(raw, account_identifier):
    name = raw.get("Name")
    region = raw.get("Region") or "global"
    is_public = _bucket_is_public(raw)
    return NormalizedAsset(
        resource_id=f"arn:aws:s3::{name}",
        cloud_provider="aws",
        account_identifier=account_identifier,
        asset_type="aws.s3.bucket",
        region=region,
        is_public=is_public,
        encryption_enabled=bool(raw.get("Encryption", {}).get("enabled")),
        network_exposure="public" if is_public else "private",
        raw=raw,
    )
```

Normalized S3 result

```
{
  "resource_id": "arn:aws:s3:::customer-exports",
  "cloud_provider": "aws",
  "account_identifier": "123456789012",
  "asset_type": "aws.s3.bucket",
  "region": "eu-west-1",
  "tags": {"env": "prod", "owner": "data"},
  "is_public": true,
  "encryption_enabled": true,
  "network_exposure": "public",
  "relationships": [{"type": "BELONGS_TO", "target_id": "123456789012"}]
}
```

EC2 normalization builds an account-and-region-qualified ARN, derives public/vpc exposure from the public IP, and creates relationships to security groups and instance profiles. Security-group normalization derives world-open ports from ingress rules.

Outcome: rules do not need to understand every AWS response shape. They evaluate a unified representation and can work across AWS, Azure, and GCP as normalizers are added.

6. Scan job, persistence, and asset lifecycle

InventoryService opens and starts a ScanJob before it persists the work. Each resource is normalized independently; a malformed record creates a ScanError but does not abort the entire scan.

```
job = ScanJobRepository.create(session, account, scan_type)
ScanJobRepository.start(job)

for source_type, raw in resources:
    fn = get_normalizer(source_type)
    try:
        normalized_asset = fn(raw, account_identifier)
        normalized.append(normalized_asset)
    except Exception as exc:
        ScanJobRepository.add_error(session, job, str(exc),
                                   resource_type=source_type)

stats = AssetRepository.upsert_assets(
    session, account, job, normalized,
    authoritative_scopes=effective_scopes,
)
ScanJobRepository.complete(job, stats)
```

Database records created or updated

- cloud_accounts: provider/account identity, role ARN, ExternalId, active status.
- scan_jobs and scan_errors: queued/running/completed/failed lifecycle, counts, and individual errors.
- assets: raw record, normalized posture projection, tags/properties/relationships, active state, timestamps, risk score.
- asset_events: created, configuration change, reactivated, or deleted events for audit/drift history.

```
UNIQUE (account_id, cloud_provider, resource_id)

# Safe retirement rule
# full import: retire unseen assets
# live collection: retire only assets whose (asset_type, region)
# is in authoritative_scopes and was not seen
```

Outcome: the current inventory is accurate without destroying history. A disappearing resource becomes inactive; it can be reactivated on a later successful scan, and its prior evidence remains auditable.

6a. Data contracts, database queries, and tenant isolation

The system deliberately separates provider evidence from the projected fields the UI and rules query. This lets the product add a new cloud type without changing every downstream screen or rule.

- Raw tuple: (source_type, raw_dict) exactly as collected from a provider, plus region and selected enrichment.
- NormalizedAsset: common resource_id, provider, account identifier, type, region, tags, posture booleans, raw evidence, properties, and relationships.
- ScanJob / ScanError: durable status, counts, timestamps, partial/failure context, and a per-operation diagnostic record.
- Asset / AssetEvent: the active inventory row plus a historical event when a resource is created, changed, reactivated, or retired.
- Finding / Issue / ComplianceSnapshot: posture result, correlated risk path, and framework score/control history tied back to the account and resource.

How inventory queries stay scoped and fast

All inventory reads start from the connected CloudAccount and filter the requested provider/account scope before optional asset-type, region, active-state, tag, or risk filters. The database has account/provider/resource identity and common query indexes; PostgreSQL is the intended production backend.

```
# Conceptual asset read used by the API
query = select(Asset).where(
    Asset.account_id == account.id,
    Asset.cloud_provider == provider,
    Asset.is_active.is_(True),
)
query = apply_region_type_tag_and_risk_filters(query)
rows = session.execute(query.order_by(Asset.risk_score.desc()))
```

Security property: a resource ID is not globally unique. The durable unique key includes account_id and provider, which prevents the same AWS ARN-like string in a different tenant/account from overwriting another customer's evidence.

7. Findings, attack paths, and compliance

After persistence, the orchestrator evaluates the active assets. The rules engine creates/updates posture findings; the issue engine correlates multiple conditions into attack paths; compliance maps open findings to frameworks.

```
findings = service.evaluate_findings(provider, account)
issues = service.evaluate_issues(provider, account)
service.snapshot_compliance(
    provider, account, ["CIS", "SOC2", "NIST", "PCI-DSS"]
)
```

Representative finding output

```
{
  "rule_id": "PUBLIC_S3_BUCKET",
  "severity": "high",
  "status": "open",
  "resource_id": "arn:aws:s3::customer-exports",
  "why": "Bucket policy or ACL permits public access",
  "remediation": "Enable S3 Block Public Access and remove public grants",
  "risk_score": 4.5,
  "exposure": "public"
}
```

Representative correlated issue

```
{
  "issue_type": "PUBLIC_COMPUTE_TO_ADMIN",
  "severity": "critical",
  "risk_score": 92.0,
  "resource_id": "arn:aws:ec2::instance/i-0ab123",
  "path": ["internet", "i-0ab123", "sg-01", "admin-role"],
  "status": "open"
}
```

A finding is one misconfiguration. An issue is a toxic combination or reachable route. Both have lifecycle reconciliation: still present = refresh; fixed = resolved; returned = reopened.

Outcome: a CISO sees a prioritized risk story rather than an unranked list of cloud settings. Compliance snapshots retain score and control history for audit and drift analysis.

8. Other CSPM product paths

- CWPP / vulnerabilities: EC2 package inventory through SSM and container image scanning through Trivy/ECR are persisted as vulnerabilities, enriched by the NVD catalog, and prioritized with KEV/EPSS maturity.
- DSPM: Data-store classification adds sensitivity labels and taxonomies. Finding risk can be elevated where public exposure and sensitive data overlap.
- Runtime: The internal runtime-events endpoints persist enriched eBPF/sensor events. These can mark attack-path issues as actively exploited.
- IaC: Terraform-plan and CloudFormation content can be scanned before deployment. This is a stateless pre-deploy path rather than cloud-inventory persistence.
- Graph: Normalized relationships become graph edges and a security graph used to enumerate reachable attack paths across an account or fleet.

9. API outcomes for the UI

POST /api/inventory/accounts	-> connected account row
POST /api/inventory/accounts/{id}/verify	-> assume/read diagnostics
POST /api/inventory/accounts/{id}/scan	-> one completed account scan
POST /api/inventory/scan-all	-> per-account fleet results
GET /api/inventory	-> paginated assets and filters
GET /api/inventory/scan-jobs/{job_id}	-> scan lifecycle/counts/errors
GET /api/inventory/findings	-> prioritized posture findings
GET /api/inventory/issues	-> correlated attack-path issues
GET /api/inventory/graph	-> security graph + paths
GET /api/inventory/compliance/*	-> snapshot/history/drift

When ODINEYES_ADMIN_TOKEN is configured, write/scan/evaluation endpoints require Authorization: Bearer <token>. Local development remains open if no token is configured.

The frontend should present scan status, assets found/changed/deleted, errors, and the partial flag. A partial result is valuable security information: it tells the operator which cloud scope still needs permissions or a retry.

What the UI should communicate at each state

- Connected but unscanned: show the provider, account identifier, role verification result, requested regions, and a clear Run scan action.
- Queued/running: show job start time, account, scan type, live counts where available, and prevent a duplicate same-account action.
- Completed: lead with critical/open findings, changed assets, compliance movement, and a direct link to raw evidence and remediation.
- Partial: keep last known assets and findings visible; list the failed API, region, and permission/remediation needed instead of showing a misleading zero.
- Failed: distinguish an authentication/configuration failure from an internal processing error; retain the prior successful inventory as historical context.

10. One resource end-to-end: actual outcome

1. AWS S3 ListBuckets finds customer-exports
2. Enrichment finds Region=eu-west-1, public policy, encryption enabled
3. Collector emits ("aws.s3.bucket", raw_dict)
4. normalize_s3_bucket emits a NormalizedAsset
5. AssetRepository upserts assets row and writes created/config_change event
6. Rule PUBLIC_S3_BUCKET opens or refreshes a high finding
7. DSPM label, if present, increases risk for public + sensitive data
8. Compliance mapper records affected CIS/SOC2/NIST/PCI controls
9. UI shows the bucket in Inventory, Finding detail, Compliance, and Graph

If the next successful S3 listing no longer contains the bucket, the asset is soft-deleted and the finding can resolve. If the S3 API was denied or truncated instead, neither event occurs: the prior asset and finding remain until evidence is trustworthy.

11. Scaling and Go boundary

The current AWS collector is primarily I/O-bound and already uses adaptive retry plus bounded threads. The next production step is a durable job queue and a database-backed distributed account lock, not a language rewrite.

Recommended future boundary

Go collector worker

- > versioned raw-resource batches + coverage/error report
- > existing Python normalization, rules, persistence API

Keep tenant identity, finding lifecycle, compliance semantics, and database ownership in one service until the event contract is stable.

Go becomes useful when measured fleet scale shows Python process overhead, connection reuse, streaming, or memory pressure dominating. It should optimize collection mechanics, while the CSPM decision model remains consistent and testable.